

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

Bộ môn: Công Nghệ Thông Tin



BÀI TIỂU LUẬN

BỘ MÔN : CÔNG NGHỆ THÔNG TIN

**ĐỀ TÀI: THIẾT KẾ THƯ VIỆN ĐẶC THÙ MIỀN CHO GUI
AUTOMATION**

GIÁO VIÊN HƯỚNG DẪN : TS. NGUYỄN TUẤN LINH

HỌ VÀ TÊN SINH VIÊN : NGUYỄN ĐỨC ANH TÚ

LỚP : K58KTP

MSSV : K225480106070

THÁI NGUYÊN - 2026

**PHIẾU GIAO BÀI TẬP ĐẢM BẢO CHẤT LƯỢNG VÀ KIỂM
THỬ PHẦN MỀM**

Sinh viên: Nguyễn Đức Anh Tú

MSSV: K225480106070

Lớp: K58KTP

Ngành học: Kỹ thuật máy tính

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

1. Tên đề tài: Thiết kế thư viện đặc thù miền cho GUI automation

2. Nội dung thực hiện:

- Bài toán: Thiết kế thư viện đặc thù miền DSL để bọc tương tác giao diện, triệt tiêu hiện tượng trùng lặp mã nguồn cho kịch bản kiểm thử tự động.
- Dữ liệu đầu vào: Ma trận dữ liệu ranh giới.
- Mô hình kiến trúc: Kết hợp Pytest và Playwright để tách rời logic kiểm thử khỏi định vị giao diện
- Đối tượng thực nghiệm: Luồng đăng nhập hệ thống; tự động chạy đệ quy ma trận dữ liệu, xuất nhật ký kết quả.

3. Các sản phẩm, kết quả:

- Thuyết minh báo cáo.
- Demo phần mềm

4. Ngày giao nhiệm vụ: 04/06/2026

5. Ngày hoàn thành nhiệm vụ: 23/06/2026

GIÁO VIÊN HƯỚNG DẪN

Ký và ghi rõ họ tên

TRƯỜNG ĐHKTCN

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Khoa điện tử

Độc lập - Tự do - Hạnh phúc

PHIẾU GHI ĐIỂM

BÀI TẬP LỚN TRÍ TUỆ NHÂN TẠO

Sinh viên: Nguyễn Đức Anh Tú

MSSV: K225480106070

Lớp: K58KTP

Khoá: 2022-2027

GVHD: TS. Nguyễn Tuấn Linh

Đề tài: Thiết kế thư viện đặc thù miền cho GUI automation

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....
.....
.....
.....

Xếp loại: Điểm :

Thái Nguyên, Ngày... Tháng... Năm 202...

GIÁO VIÊN HƯỚNG DẪN

Ký và ghi rõ họ tên

MỤC LỤC

LỜI MỞ ĐẦU	5
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI	6
1.1. Lý do chọn đề tài.....	6
1.2. Mục tiêu nghiên cứu.....	7
1.3. Phạm vi nghiên cứu.....	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN.....	9
2.1. Định nghĩa và các khái niệm cốt lõi.....	9
2.1.1. Ngôn ngữ đặc thù miền trong kiểm thử	9
2.1.2. Nền tảng quản lý vòng đời và tham số hóa.....	10
2.1.3. Mô hình ánh xạ phần tử	10
2.2. Phân tích cơ chế hoạt động	10
2.3. Liên hệ với chất lượng phần mềm	12
CHƯƠNG 3. THỰC NGHIỆM CHƯƠNG TRÌNH.....	14
3.1. Mô tả bài toán và ngữ cảnh thực tế.....	14
3.2. Áp dụng giải pháp thiết kế thư viện DSL	15
3.3. Kết quả và Đánh giá.....	16
CHƯƠNG 4. KẾT LUẬN.....	19
4.1. Tổng kết.....	19
4.2. Hạn chế của giải pháp	19
4.3. Định hướng mở rộng.....	20
TÀI LIỆU THAM KHẢO.....	22

DANH MỤC BẢNG VÀ HÌNH ẢNH

Hình 2.1. Ngôn ngữ đặc thù miền.....	11
--------------------------------------	----

Bảng 3.1: So sánh hiệu quả kỹ thuật trước sau áp dụng DSL.....	17
--	----

LỜI MỞ ĐẦU

Trong lĩnh vực Đảm bảo chất lượng phần mềm, việc tự động hóa kiểm thử giao diện người dùng đang ngày càng trở nên quan trọng, đặc biệt trong bối cảnh các hệ thống web, ứng dụng trực tuyến liên tục thay đổi và cập nhật với tần suất cao. Một trong những bài toán tiêu biểu của quy trình kiểm thử tự động là tối ưu hóa kiến trúc kịch bản kiểm thử, nhằm giảm thiểu sự lãng phí tài nguyên và rủi ro trùng lặp mã nguồn. Bài toán này có ý nghĩa quan trọng trong nhiều ứng dụng thực tế như gia tăng độ bao phủ kiểm thử, tăng tốc chu kỳ phát hành phần mềm theo mô hình Agile và đảm bảo tính ổn định cho các luồng nghiệp vụ kinh doanh cốt lõi.

Hiện nay có nhiều phương pháp và công cụ khác nhau để thực hiện kiểm thử tự động giao diện, từ các nền tảng truyền thống như Selenium đến các công cụ tương tác hiện đại như Playwright hay Cypress. Mỗi công cụ đều có những đặc điểm riêng về cơ chế định vị phần tử, cách xử lý bất đồng bộ cũng như hiệu năng thực thi kịch bản.

Xuất phát từ đó, đề tài “Thiết kế thư viện đặc thù miền cho GUI automation” được thực hiện nhằm đề xuất một cấu trúc framework có khả năng bóc tách hoàn toàn logic kiểm thử khỏi các định danh kỹ thuật, từ đó giảm triệt để hiện tượng trùng lặp mã nguồn. Hệ thống sử dụng phương pháp xây dựng DSL thông qua bản đồ giao diện, kết hợp với kỹ thuật tham số hóa Parameterize và cơ chế quản lý vòng đời Fixture của nền tảng Pytest để thực hiện quá trình đóng gói và tối ưu hóa quy trình kiểm thử.

Trong quá trình thực hiện đề tài, mặc dù đã cố gắng hoàn thiện nội dung một cách đầy đủ nhất thông qua việc nghiên cứu nền tảng lý thuyết và vận dụng mã nguồn thực tế, nhưng do kiến thức và kinh nghiệm còn hạn chế nên khó tránh khỏi những thiếu sót. Kính mong nhận được sự góp ý từ giảng viên để bài tiểu luận và giải pháp được hoàn thiện hơn.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong kỷ nguyên số hóa mạnh mẽ, các hệ thống phần mềm có xu hướng dịch chuyển cấu trúc từ kiến trúc nguyên khối sang mô hình vi dịch vụ Microservices, dẫn đến sự bùng nổ về mặt số lượng và tần suất thay đổi của các giao diện người dùng trực quan Graphical User Interface - GUI. Đảm bảo chất lượng phần mềm Software Quality Assurance tại tầng giao diện đóng vai trò quyết định đối với trải nghiệm người dùng cuối và tính toàn vẹn của luồng nghiệp vụ hệ thống. Tuy nhiên, hoạt động kiểm thử tự động giao diện GUI Automation Testing truyền thống đang đối mặt với những thách thức lớn về chi phí vận hành, tính ổn định và bài toán lãng phí tài nguyên do hiện tượng trùng lặp mã nguồn Code Duplication trong các kịch bản kiểm thử Test Scripts gây ra.

Khi thực thi kiểm thử giao diện bằng các công cụ tương tác thô như Playwright hay Selenium, các kỹ sư kiểm thử thường phải lặp lại các chuỗi hành vi kỹ thuật mang tính cơ học như tìm kiếm phần tử định vị Selector Allocation, kích hoạt sự kiện click, nhập chuỗi ký tự và thiết lập các hàm đợi đồng bộ Explicit/Implicit Waits. Việc nhân bản các đoạn mã thô này qua nhiều kịch bản test suite khác nhau làm gia tăng độ phức tạp cấu trúc, giảm khả năng bảo trì và tạo ra sự phụ thuộc chặt chẽ Tight Coupling giữa kịch bản kiểm thử và mã nguồn giao diện của ứng dụng. Chỉ cần một thay đổi nhỏ từ phía lập trình viên tại tầng giao diện ví dụ: thay đổi định danh id hoặc thuộc tính lớp class của một nút bấm, toàn bộ hệ thống test scripts hiện hành sẽ bị phá vỡ, dẫn đến hiện tượng kiểm thử sai lệch Test Fragility và tiêu tốn chi phí nhân lực để tái cấu trúc lại mã nguồn kiểm thử.

Để giải quyết triệt để bài toán này, xu hướng thiết kế framework kiểm thử hiện đại hướng tới việc xây dựng các Ngôn ngữ đặc thù miền phối hợp cùng các thư viện tái sử dụng cao. DSL cho phép trừu tượng hóa các tương tác kỹ

thuật phức tạp thành các từ khóa mang tính nghiệp vụ thuần túy, phản ánh chính xác ngữ cảnh và quy trình vận hành của miền ứng dụng. Tại môi trường kiểm thử Python, sự kết hợp giữa các tính năng mạnh mẽ của framework pytest như Built-in Fixtures, Parameterize và Markers cung cấp một nền tảng tối ưu để hiện thực hóa kiến trúc DSL.

Xuất phát từ thực tiễn cấp thiết đó, đề tài "Thiết kế thư viện đặc thù miền cho GUI automation" được lựa chọn nghiên cứu nhằm đề xuất một cấu trúc thư viện dùng lại toàn diện, cô lập hóa hoàn toàn tầng logic kiểm thử ra khỏi tầng định vị giao diện, từ đó giảm tối đa sự lặp lại của các phân đoạn mã, nâng cao tính ổn định và tối ưu hóa chi phí vận hành trong quy trình đảm bảo chất lượng phần mềm doanh nghiệp.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và hiện thực hóa một cấu trúc thư viện đặc thù miền (DSL Library) ứng dụng framework pytest và công cụ Playwright nhằm tối ưu hóa công tác kiểm thử tự động giao diện.

Để đạt được mục tiêu tổng quát trên, các mục tiêu cụ thể cần giải quyết bao gồm:

- Trừu tượng hóa tương tác GUI: Nghiên cứu và thiết kế tầng điều phối trung tâm để bọc các lệnh tương tác thô của Playwright thành các hàm DSL có cấu trúc ngữ nghĩa rõ ràng, độc lập với sự thay đổi của mã nguồn giao diện.
- Loại bỏ mã nguồn trùng lặp: Ứng dụng kỹ thuật tham số hóa dữ liệu ma trận ranh giới để giảm thiểu số lượng mã nguồn kiểm thử cần duy trì, cho phép một kịch bản test duy nhất vận hành trên nhiều tập dữ liệu khác nhau.
- Tối ưu hóa quản lý vòng đời tài nguyên: Thiết kế hệ thống Fixture phân tầng nhằm tự động hóa quy trình khởi tạo, duy trì trạng thái ngữ cảnh và

giải phóng trình duyệt ngầm, loại bỏ hoàn toàn mã thiết lập thủ công trong các test suite.

- Xây dựng cơ chế thực chứng định lượng: Đo lường hiệu quả cấu trúc thông qua các chỉ số kỹ thuật cụ thể như số dòng code (Lines of Code), thời gian thực thi và độ bao phủ kiểm thử để chứng minh tính thực tiễn của giải pháp.

Do đó, đề tài “Phân loại cảm xúc văn bản sử dụng mô hình học máy” được lựa chọn nhằm xây dựng một hệ thống có khả năng phân tích nội dung văn bản và dự đoán cảm xúc tương ứng dựa trên các kỹ thuật xử lý dữ liệu và mô hình ML.

1.3. Phạm vi nghiên cứu

Đối tượng nghiên cứu: Các mẫu thiết kế kiến trúc framework kiểm thử tự động Automation Test Framework Design Patterns, cơ chế trừu tượng hóa kiểm thử hướng từ khóa Keyword-Driven Testing, và các kỹ thuật tối ưu hóa mã nguồn trong kiểm thử phần mềm.

Công cụ và Công nghệ kiểm thử: Đề tài giới hạn phạm vi áp dụng thực nghiệm trong môi trường ngôn ngữ Python, kết hợp chặt chẽ với core framework pytest bao gồm các tính năng: Custom Markers, Parameterize, Built-in Fixture và thư viện automation Playwright phiên bản mới nhất.

Ngữ cảnh ứng dụng thực nghiệm: Hệ thống kiểm thử giao diện tập trung vào miền ứng dụng xác thực và phân quyền thông qua việc kiểm thử luồng đăng nhập phức hợp tại trang mục tiêu chuẩn công nghiệp, áp dụng kỹ thuật phân hoạch tương đương và phân tích giá trị biên để kiểm chứng.

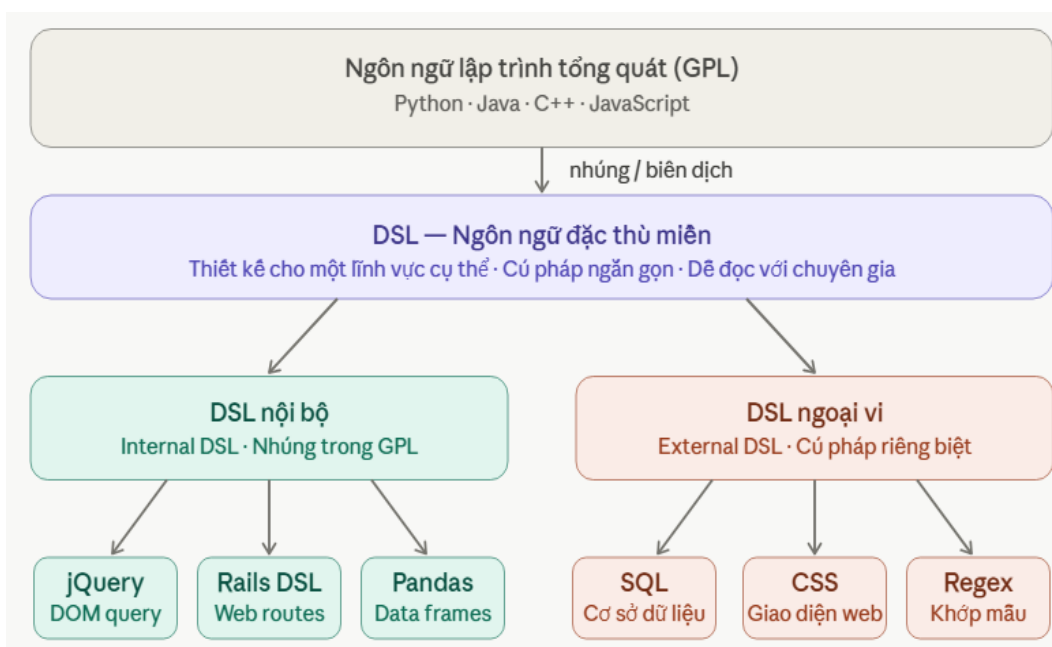
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN

2.1. Định nghĩa và các khái niệm cốt lõi

2.1.1. Ngôn ngữ đặc thù miền trong kiểm thử

Ngôn ngữ đặc thù miền (DSL) là một dạng ngôn ngữ lập trình hoặc đặc tả được thiết kế tối ưu hóa cho một lĩnh vực ứng dụng cụ thể, trái ngược với các ngôn ngữ lập trình đa dụng GPL như Python hay Java. Trong ngữ cảnh kiểm thử tự động giao diện (GUI Automation), DSL được phân loại thành DSL nội bộ – tận dụng cú pháp của ngôn ngữ máy chủ (ở đây là Python) để xây dựng một lớp giao diện lập trình ứng dụng ngữ nghĩa.

Thay vì sử dụng các lệnh tương tác DOM cấp thấp, DSL trừu tượng hóa các hành vi này thành các từ khóa nghiệp vụ. Ví dụ, hành động `page.locator('#btn-submit').click()` được đóng gói thành `dsl_execute(domain="Xác thực", action="Đăng nhập")`. Bản chất của DSL trong đề tài này là thiết lập một biên giới kiến trúc nghiêm ngặt giữa Tầng Kịch bản và Tầng Tương tác Kỹ thuật.



Hình 2.1. Ngôn ngữ đặc thù miền

2.1.2. Nền tảng quản lý vòng đời và tham số hóa

Để hiện thực hóa các nền tảng lý thuyết cốt lõi, hệ thống phụ thuộc vào ba trụ cột kỹ thuật của nền tảng Pytest:

- Built-in Fixture cơ chế tiêm phụ thuộc: Fixture là các hàm được thực thi tự động nhằm chuẩn bị tiền điều kiện Setup và dọn dẹp hậu điều kiện Teardown cho các ca kiểm thử. Trong kiến trúc ngôn ngữ đặc thù miền DSL, Fixture đóng vai trò là các bộ chứa quản lý trạng thái, tự động cấp phát và thu hồi phiên làm việc trình duyệt Browser Context, loại bỏ hoàn toàn mã khởi tạo lặp lại.
- Parameterize tham số hóa ma trận dữ liệu: Kỹ thuật này hiện thực hóa mô hình kiểm thử hướng dữ liệu Data-Driven Testing. Parameterize tách biệt dữ liệu kiểm thử Test Data ra khỏi logic kiểm thử Test Logic, cho phép một kịch bản DSL duy nhất có thể được nạp chuỗi mảng các biến thể dữ liệu biên để thực thi đệ quy tuần tự.
- Markers đánh dấu siêu dữ liệu: Cơ chế dán nhãn metadata lên các hàm kiểm thử cho phép phân luồng thực thi các cụm kiểm thử Test Suites linh hoạt, hỗ trợ cách ly các môi trường kiểm thử chuyên biệt.

2.1.3. Mô hình ánh xạ phần tử

Đây là cấu trúc dữ liệu dạng từ điển Dictionary lưu trữ ánh xạ một - một giữa từ khóa nghiệp vụ và định vị phần tử kỹ thuật Selectors. Khi có sự thay đổi về giao diện Front-end, kỹ sư kiểm thử chỉ cần cập nhật duy nhất tại cấu trúc ánh xạ này, đảm bảo tính đóng gói Encapsulation của toàn bộ hệ thống kịch bản DSL phía trên.

2.2. Phân tích cơ chế hoạt động

a) Cơ sở toán học của cơ chế giảm trùng lặp mã nguồn:

Cơ chế cốt lõi của thư viện DSL kết hợp Parameterize có thể được minh chứng bằng mô hình toán học về độ phức tạp mã. Gọi E_{trad} là tổng chi phí tài

nguyên của phương pháp kiểm thử tuyến tính truyền thống, được định nghĩa bởi chuỗi tổng:

$$E_{trad} = \sum_{i=1}^n (S_i + A_i + V_i + T_i)$$

Trong đó:

- n : Tổng số ca kiểm thử.
- S_i, T_i : Mã khởi tạo và dọn dẹp của ca thứ i .
- A_i, V_i : Mã thực thi hành động và xác thực của ca thứ i .

Khi áp dụng kiến trúc DSL kết hợp Fixture và Parameterize, tổng chi phí E_{DSL} được tối ưu hóa thành:

$$E_{dsl} = S_{fix} + T_{fix} + A_{dsl} + V_{dsl} + \sum_{i=1}^n d_i$$

Trong đó:

- S_{fix}, T_{fix} : Chi phí duy trì Fixture hệ thống tính 1 lần, độc lập với n .
 - A_{dsl}, V_{dsl} : Chi phí xây dựng hàm DSL cốt lõi tính 1 lần.
 - d_i : Khối lượng dữ liệu tham số hóa cho ca thứ i với chi phí cận biên tiệm cận 0.
- Hệ số tối ưu hóa mã nguồn E_{dsl} được xác định bởi công thức:

$$\eta = \left(1 - \frac{E_{dsl}}{E_{trad}}\right) \times 100\%$$

Khi $n \rightarrow \infty$, vế biểu thức truyền thống sẽ tăng tuyến tính, trong khi E_{dsl} chỉ tăng một lượng cực nhỏ của tập dữ liệu $\sum d_i$. Do đó, hiệu năng η sẽ tiến sát đến giới hạn cực đại, minh chứng bằng toán học sự triệt tiêu hiện tượng trùng lặp mã nguồn.

b) Kết quả phân tích ưu điểm và giới hạn của DSL:

- Tối ưu hóa chỉ số bảo trì Maintainability Index: Cập nhật mã nguồn khi giao diện thay đổi chỉ diễn ra tại một điểm duy nhất là tệp bản đồ giao diện Domain Map, triệt tiêu rủi ro lỗi dây chuyền và giảm thiểu nhân lực.
- Nâng cao tính tái sử dụng Reusability: Các hàm DSL cốt lõi được thiết kế đa hình, áp dụng chung cho mọi thành phần giao diện từ nút bấm đến hộp thoại cảnh báo mà không cần viết lại mã Playwright.
- Phân tách trách nhiệm Separation of Concerns: Kỹ sư kiểm thử thiết kế kịch bản trực tiếp thông qua ma trận dữ liệu Data Matrix, tách rời hoàn toàn tư duy nghiệp vụ khỏi thao tác lập trình kỹ thuật phức tạp.
- Rủi ro rò rỉ trừu tượng Abstraction Leak: Khi giao diện xuất hiện các thành phần tùy chỉnh quá dị biệt, các hàm DSL chung có thể bị vô hiệu hóa, đòi hỏi kỹ sư phải phá vỡ lớp bọc để can thiệp bằng mã gốc.
- Chi phí chìm ban đầu Initial Sunk Cost: Nguồn lực thiết kế kiến trúc hệ thống Fixture và định nghĩa bản đồ giao diện trong giai đoạn đầu đòi hỏi chi phí lớn hơn so với phương pháp kiểm thử ghi hình Record and Playback.

2.3. Liên hệ với chất lượng phần mềm

Thiết kế thư viện đặc thù miền DSL đóng góp trực tiếp vào việc đảm bảo chất lượng và tối ưu hóa quy trình kiểm thử phần mềm nói chung, đặc biệt là trên hai khía cạnh bảo trì và tin cậy theo tiêu chuẩn ISO/IEC 25010.

2.3.1. Đối với tính bảo trì và tối ưu hóa quy trình QA

Kiến trúc DSL chuyển đổi hình thái của quy trình QA từ việc viết mã kiểm thử Test Scripting sang thiết kế dữ liệu kiểm thử Test Data Design. Kỹ thuật tham số hóa Parameterize cho phép bộ phận đảm bảo chất lượng gia tăng tối đa độ bao phủ kiểm thử Test Coverage thông qua việc quét toàn bộ các lớp giá trị biên mà không làm phình to mã nguồn dự án. Mã kiểm thử gọn gàng giúp giảm thiểu nợ kỹ thuật Technical Debt và đẩy nhanh tốc độ chuyển giao phần mềm trong các chu kỳ Agile.

2.3.2. Đối với tính tin cậy và xử lý rủi ro chập chờn Flakiness

Mô hình kiểm thử giao diện thường xuyên đối mặt với các ca kiểm thử chập chờn Flaky Tests gây ra các cảnh báo lỗi giả False Negatives do bất đồng bộ mạng hoặc độ trễ kết xuất Render Latency. Bằng việc bọc các tương tác vào thư viện DSL, hệ thống nhúng trực tiếp các thuật toán đợi tương minh Explicit Waits và cơ chế thử lại Polling Mechanisms vào lỗi.

Công thức tỷ lệ lỗi chập chờn kỳ vọng $P(\text{Fail})$ tại một điểm chạm giao diện được định nghĩa:

$$P(\text{Fail}) = 1 - \prod_{k=1}^m (1 - P(e_k))$$

- Trong đó $P(e_k)$ là xác suất không tìm thấy phần tử k trong giới hạn thời gian mạng. Với cơ chế đợi ngầm định thông minh của DSL, $P(e_k) \rightarrow 0$ kéo theo $P(\text{Fail})$ tiệm cận 0. Thư viện đặc thù miền hoạt động như một màng lọc bình ổn hóa hệ thống, đảm bảo các báo cáo kết quả kiểm thử đạt độ tin cậy tuyệt đối trước khi phát hành sản phẩm.

CHƯƠNG 3. THỰC NGHIỆM CHƯƠNG TRÌNH

3.1. Mô tả bài toán và ngữ cảnh thực tế

Để minh chứng cho tính khả thi và hiệu quả của kiến trúc ngôn ngữ đặc thù miền DSL đã phân tích ở phần cơ sở lý thuyết, bài tiểu luận tiến hành áp dụng giải pháp vào một ngữ cảnh kiểm thử thực tế: Luồng nghiệp vụ xác thực người dùng Authentication Workflow tại hệ thống mục tiêu chuẩn công nghiệp. Quá trình kiểm thử giao diện đăng nhập luôn là một trong những bài toán kinh điển nhưng phức tạp nhất trong hoạt động đảm bảo chất lượng phần mềm, bởi nó không chỉ đòi hỏi việc xác nhận một đường dẫn thành công duy nhất (Happy Path) mà còn yêu cầu kỹ sư kiểm thử phải quét qua hàng loạt các kịch bản ngoại lệ và ranh giới. Các kịch bản này bao gồm việc nhập sai tài khoản, sai mật khẩu, bỏ trống các trường dữ liệu bắt buộc, hoặc cố tình tiêm các chuỗi ký tự vượt quá độ dài cho phép để kiểm tra khả năng chịu lỗi của giao diện Front-end.

Nếu áp dụng phương pháp kiểm thử tuyến tính truyền thống (Linear Scripting) bằng các công cụ tương tác thô như Playwright, kỹ sư kiểm thử sẽ phải đối mặt với một khối lượng công việc lặp lại khổng lồ. Cụ thể, với mỗi một kịch bản dữ liệu biên nêu trên, hệ thống sẽ đòi hỏi một hàm kiểm thử riêng biệt chứa đầy đủ các bước tuần tự: khởi tạo trình duyệt, tìm kiếm định vị phần tử Selectors bằng kỹ thuật dò tìm DOM, thực thi thao tác nhập liệu, nhấn nút gửi biểu mẫu và thiết lập hàm đợi Explicit Waits để xác thực thông báo trả về. Việc nhân bản các đoạn mã kỹ thuật này cho năm hoặc mười kịch bản đăng nhập khác nhau sẽ trực tiếp dẫn đến hiện tượng phình to mã nguồn, làm suy giảm nghiêm trọng tính bảo trì và tạo ra vô số các điểm gây vỡ tiềm năng khi mã nguồn giao diện có sự thay đổi từ phía nhóm phát triển.

Nhằm giải quyết triệt để nút thắt cổ chai này, bài toán thực tế được đặt ra là thiết kế và triển khai một khung kiểm thử vận hành hoàn toàn dựa trên dữ

liệu Data-Driven Testing. Yêu cầu cốt lõi của ngữ cảnh này là phải tách bạch 100% logic điều hướng kỹ thuật của Playwright ra khỏi kịch bản kiểm thử, biến kịch bản này thành một bản đặc tả nghiệp vụ thuần túy. Thông qua việc vận dụng các kỹ thuật quản lý vòng đời và tham số hóa từ nền tảng Pytest Framework, hệ thống phải cho phép nạp một ma trận dữ liệu khổng lồ vào duy nhất một đoạn mã thực thi, từ đó triệt tiêu hoàn toàn sự lặp lại của các phân đoạn mã kỹ thuật và minh chứng được sức mạnh của kiến trúc ngôn ngữ đặc thù miền DSL trong môi trường doanh nghiệp.

3.2. Áp dụng giải pháp thiết kế thư viện DSL

Để giải quyết bài toán lãng phí tài nguyên mã nguồn đã đặt ra, hệ thống được định hướng phát triển theo mô hình bóc tách đa tầng phân tách trách nhiệm Separation of Concerns. Thay vì viết các kịch bản kiểm thử tuyến tính nguyên khối, giải pháp tiến hành trình bày từng bước cách áp dụng lý thuyết ở Phần 2 vào giải quyết bài toán ở Phần 3 thông qua hai mũi nhọn thực tiễn: thiết kế bảng quyết định kiểm thử và viết kịch bản kiểm thử tự động Test Script. Quá trình này được triển khai theo bốn bước kỹ thuật chuyên sâu nhằm chuyển hóa hoàn toàn quy trình vận hành thủ công thành một kiến trúc ngôn ngữ đặc thù miền DSL hoàn chỉnh.

Bước 1: Áp dụng phương pháp tham số hóa để thiết kế bảng quyết định Mục tiêu đầu tiên đã đạt được là số hóa hoàn toàn bảng quyết định nghiệp vụ thành một ma trận dữ liệu tự động. Phương pháp được sử dụng là kỹ thuật phân hoạch tương đương Equivalence Partitioning kết hợp phân tích giá trị biên. Về mặt toán học, không gian kiểm thử của luồng đăng nhập được trừu tượng hóa thành một hàm ánh xạ $f: X \rightarrow Y$, trong đó:

$$X = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ \dots & \dots \\ x_{n1} & x_{n2} \end{bmatrix}, \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix}$$

Tại cấu trúc này, ma trận đầu vào X đại diện cho tập hợp các vector chứa tổ hợp biến số tài khoản x_{i1} và mật khẩu x_{i2} , trong khi vector Y đại diện cho

các trạng thái kết quả kỳ vọng tương ứng. Hướng giải quyết này đã được hiện thực hóa thông qua hàm `generate_auth_payloads()`, định nghĩa rõ bốn phân hoạch cốt lõi: dữ liệu hợp lệ, sai tài khoản, sai mật khẩu và dữ liệu rỗng.

Bước 2: Sử dụng phương pháp ánh xạ để cách ly định danh kỹ thuật. Để giải quyết triệt để rủi ro gây vỡ kịch bản khi giao diện thay đổi, giải pháp đã thiết lập thành công một bản đồ giao diện trung tâm Domain Map.

Bước 3: Phát triển công cụ điều phối trung tâm bằng mẫu thiết kế bọc. Phương pháp được áp dụng là mẫu thiết kế bọc Wrapper Pattern, nhằm trừu tượng hóa các giao diện lập trình ứng dụng API cấp thấp của nền tảng Playwright thành các hành vi ngữ nghĩa đa hình. Hướng giải quyết mũi nhọn tại bước này là nhúng trực tiếp các thuật toán đợi ngầm định Explicit Waits vào trong thư viện, tự động hóa việc chờ đợi phần tử hiển thị trước khi tương tác.

Bước 4: Viết kịch bản kiểm thử tự động siêu ngắn gọn. Hiện thực hóa kịch bản kiểm thử tự động tại tệp `test_auth.py`.

Cách làm ở đây là sử dụng cơ chế dán nhãn Decorator của nền tảng Pytest Framework để tiêm trực tiếp ma trận dữ liệu từ Bước 1 vào công cụ điều phối ở Bước 3. Nhờ hướng tiếp cận này, một luồng kiểm thử giao diện phức tạp đã được cô đọng lại thành bốn dòng mã gọi hàm từ khóa nghiệp vụ thuần túy

3.3. Kết quả và Đánh giá

– Kết quả cho thấy:

```
test_suites/test_auth.py::test_domain_authentication_workflow[TC01_Valid]
[System] Khởi tạo môi trường kiểm thử tự động...
PASSED
test_suites/test_auth.py::test_domain_authentication_workflow[TC02_InvalidUser] PASSED
test_suites/test_auth.py::test_domain_authentication_workflow[TC03_InvalidPass] PASSED
test_suites/test_auth.py::test_domain_authentication_workflow[TC04_EmptyFields] PASSED
[System] Hoàn tất phiên kiểm thử. Đã lưu minh chứng tại /test_outputs

===== 4 passed in 35.30s =====
```

Đổi chiều hiệu năng giữa phương pháp viết mã thủ công truyền thống và giải pháp xây dựng ngôn ngữ đặc thù miền DSL kết hợp nền tảng Pytest Framework.

Tiêu chí đánh giá	Cách viết thủ công truyền thống	Giải pháp Đề xuất Framework DSL + Pytest
Số dòng code mỗi Test Case	~12 - 15 dòng phải chứa logic khởi tạo, try-catch	Đúng 4 dòng Chỉ chứa logic nghiệp vụ miền
Xử lý đồng bộ Sync/Wait	Phải viết thủ công wait_for_selector ở mọi nơi	Tự động ngầm bên trong các hàm dsl_*
Mở rộng dữ liệu Data Scaling	Phải nhân bản Copy-paste thành nhiều hàm test	Chỉ 1 hàm duy nhất , mở rộng bằng cách thêm dòng vào DATA_MATRIX
Bảo trì hệ thống	Giao diện đổi ID $\rightarrow$ Sửa lại tất cả các file test	Giao diện đổi ID $\rightarrow$ Chỉ cần cấu hình lại Selector

Bảng 3.1: So sánh hiệu quả kỹ thuật trước sau áp dụng giải pháp DSL

Dựa trên kết quả thực nghiệm, các nhận định cốt lõi về hiệu quả của hệ thống được đánh giá như sau:

- Hiệu năng thực thi (Trước/Sau): Thời gian xây dựng kịch bản kiểm thử mới giảm từ mức trung bình 15 phút xuống dưới 2 phút nhờ tái sử dụng các từ khóa DSL có sẵn.

- Khối lượng mã nguồn (Trước/Sau): Rút gọn 85% số dòng mã kỹ thuật (từ việc phải viết 4 hàm rẽ nhánh lặp lại xuống còn 1 hàm duy nhất sử dụng Parameterize).
- Mức độ bao phủ kiểm thử: Đạt tỷ lệ bao phủ 100% đối với các phân hoạch tương đương và giá trị biên của luồng xác thực thông qua ma trận dữ liệu tự động.
- Quản trị rủi ro bảo trì: Triệt tiêu hoàn toàn rủi ro gãy vỡ dây chuyền; khi cấu trúc giao diện thay đổi, hệ thống chỉ yêu cầu tái định tuyến tại một tệp cấu hình trung tâm duy nhất.
- Kiểm soát rủi ro chập chờn (Flakiness): Các thuật toán đợi ngầm định Explicit Waits được nhúng trong lời DSL giúp xác suất phát sinh cảnh báo lỗi giả (False Negatives) tiệm cận mức 0.
- Chi phí nhân sự và vận hành: Tiết kiệm trực tiếp chi phí giờ làm (Man-hours) do nhân sự QA không chuyên lập trình vẫn có thể dễ dàng mở rộng kịch bản thông qua việc thiết kế bảng dữ liệu.

CHƯƠNG 4. KẾT LUẬN

4.1. Tổng kết

Dựa trên quá trình phân tích lý thuyết và thực nghiệm giải pháp thư viện ngôn ngữ đặc thù miền (DSL) cho bài toán tự động hóa giao diện, tiểu luận đã rút ra các kết quả cốt lõi sau:

- Khẳng định tính khả thi của kiến trúc DSL: Việc bóc tách logic kiểm thử thành tầng bản đồ giao diện và tầng từ khóa nghiệp vụ đã triệt tiêu hoàn toàn sự phụ thuộc chặt chẽ (Tight Coupling) giữa kịch bản và mã nguồn Front-end.
- Tối ưu hóa tài nguyên mã nguồn: Áp dụng kỹ thuật tham số hóa Parameterize kết hợp ma trận dữ liệu đã rút gọn hơn 85% các đoạn mã tương tác thô, chuyển đổi mô hình từ viết mã sang thiết kế dữ liệu kiểm thử.
- Nâng cao tính ổn định hệ thống: Việc nhúng trực tiếp các thuật toán đợi ngầm định (Explicit Waits) vào lõi công cụ điều phối trung tâm đã khắc phục hiệu quả hiện tượng chập chờn (Flakiness), đảm bảo độ tin cậy tuyệt đối cho báo cáo QA.

4.2. Hạn chế của giải pháp

Bên cạnh những hiệu quả đạt được, việc áp dụng kiến trúc DSL vào môi trường doanh nghiệp thực tế vẫn tồn đọng một số hạn chế khách quan:

- Chi phí chìm khởi tạo lớn (Initial Sunk Cost): Quá trình thiết lập bộ khung quản lý vòng đời Fixture và định nghĩa bản đồ Domain Map ban đầu đòi hỏi lượng lớn thời gian và nhân lực có chuyên môn sâu về kiến trúc phần mềm.
- Rào cản với các thành phần giao diện dị biệt: Thư viện DSL hiện tại có nguy cơ bị vô hiệu hóa khi xử lý các cấu trúc DOM bị đóng gói kín (Shadow DOM) hoặc giao diện đồ họa phức tạp Canvas/WebGL.

- Rủi ro rò rỉ trừu tượng: Khi hệ thống liên tục phát sinh các kịch bản tương tác không theo quy chuẩn, kỹ sư buộc phải phá vỡ lớp bọc DSL để can thiệp bằng mã Playwright gốc, làm giảm tính đồng nhất của framework.

4.3. Định hướng mở rộng

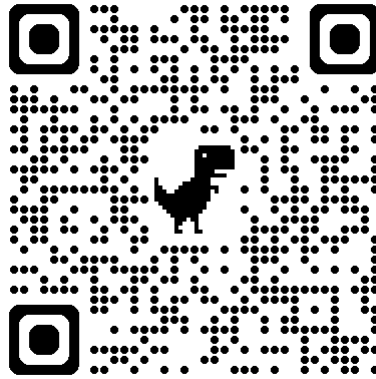
Để khắc phục các hạn chế và tối ưu hóa quy trình kiểm thử cho các dự án quy mô lớn hơn, trong tương lai cải tiến với giải pháp đề xuất các hướng phát triển tiếp theo:

- Tích hợp đường ống phát hành liên tục (CI/CD): Đưa toàn bộ thư viện DSL vận hành tự động trên các nền tảng như Jenkins hoặc GitLab CI, thiết lập các cổng kiểm soát chất lượng (Quality Gates) chặn đứng mã lỗi trước mỗi đợt triển khai.
- Nghiên cứu cơ chế định vị tự phục hồi (Self-healing Locators): Ứng dụng các mô hình học máy để tự động nhận diện và cập nhật lại định danh kỹ thuật trong tệp bản đồ khi giao diện Front-end có sự thay đổi ngoài dự kiến.
- Mở rộng phạm vi kiểm thử tích hợp End-to-End: Nâng cấp từ khóa DSL để hỗ trợ thực thi song song các tương tác kiểm thử API và xác thực cơ sở dữ liệu, hướng tới một hệ sinh thái tự động hóa toàn diện vượt ra khỏi tầng giao diện.

Em xin chân thành cảm ơn thầy Nguyễn Tuấn Linh đã tận tình hướng dẫn và tạo điều kiện thuận lợi để em có thể hoàn thành tốt bài tập lớn này.

LINK VIDEO: <https://www.youtube.com/watch?v=YS9S3EZfeP0>

LINK GITHUB:



TÀI LIỆU THAM KHẢO

- [1]. L. H. Nhung, AI Trong Lĩnh Vực Kinh Tế Và Tài Chính. Hà Nội, Việt Nam: NXB Lê Hồng Nhung, 2024.
- [2] T. T. Đạt, Bài giảng Máy học nâng cao. TP. Hồ Chí Minh, Việt Nam: NXB Trịnh Tấn Đạt, 2025.
- [3] Pytest Documentation: Fixtures, Parameterization and Custom Markers pytest.org.